Embed With Elliot: Debounce Your Noisy Buttons, Part II

If you've ever turned a rotary encoder or pushed a cursor button and had it skip a step or two, you've suffered directly from button bounce. My old car stereo and my current in-car GPS navigator both have this problem, and it drives me nuts. One button press should be one button press. How hard is that to get right?

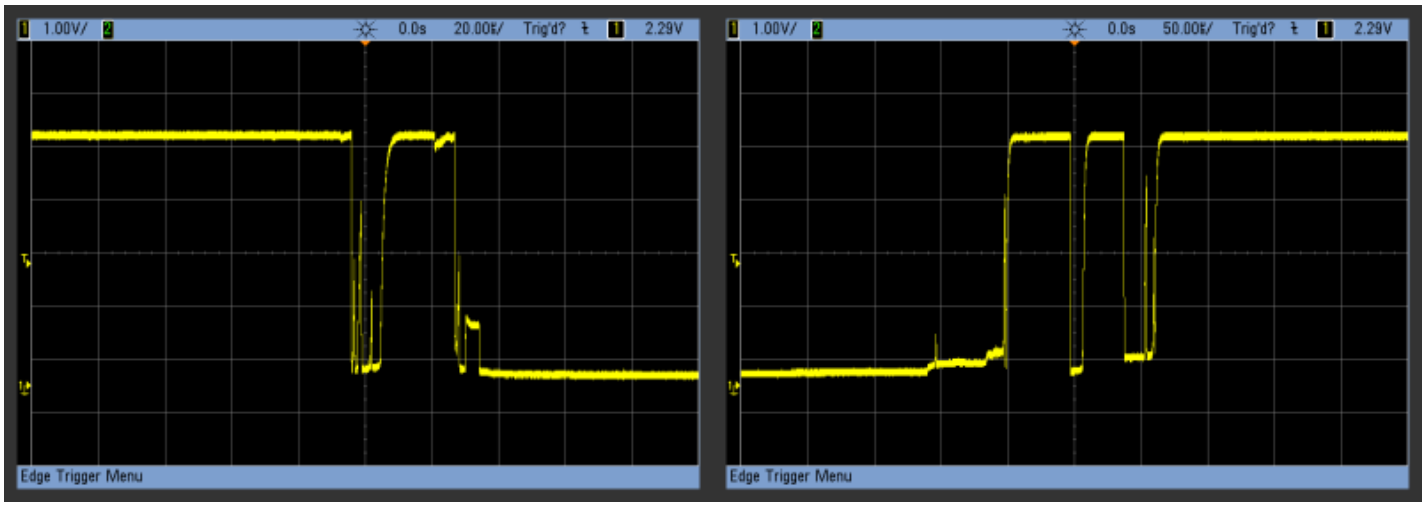
In the last session of Embed with Elliot, we looked into exactly how hard it is to get right and concluded that it wasn't actually all that bad, as long as you're willing to throw some circuitry at the problem, or accept some sluggishness in software. But engineers cut corners on hardware designs, and parts age and get dirty. Making something as "simple" as a button work with ultra-fast microcontrollers ends up being non-trivial.

And unsurprisingly, for a problem this ubiquitous, there are a myriad of solutions. Some are good, some are bad, and others just have trade-offs. In this installment, we're going to look at something special: a debouncer that uses minimal resources and is reasonably straightforward in its operation, yet which can debounce along with the best of 'em.

In short, I'll introduce you to what I think is The Ultimate Debouncer(tm)! And if you don't agree by the end of this article, I'll give you your money back.

Debouncing

First, a quick recap. When you press a physical button, two metal surfaces come into contact with each other. Between the time when they're unambiguously separate and when they're mashed together so well that they're making contact in multiple locations, there's a short period of time when the button is making and breaking contact due to the slight imperfections in the two surfaces.



The result is that, for a handful of milliseconds, the voltage level on the switch terminal is erratic and flickers between on and off. If your code detects a button press, for instance, by looking for a high to low voltage transition, it might see multiple such transitions for what you think is a single press.

You can clean this up in hardware with the classic RC filter plus inverter-with-hysteresis combo, or you can try to fix it in software with a debouncing routine. Last time, I broke down the software routines into three classes based on how they work: by delaying until the bounce is done, by smoothing (integrating) the bounce away, or by attempting to match the button pattern as a whole.

A Difference-based Debouncer

Before we get into the Ultimate, let's look at the "Penultimate Debouncer". I learned this trick from Jack Ganssle's amazing two-part treatise on button bounce (part two) many years ago, and it's almost perfect. We'll

work through his version first, because it's good enough by itself most of the time, and because it's a good starting place.

One of the debounce routines that [Jack] suggested is particularly simple. The microcontroller periodically polls the button, and stores not just its current state, but a much longer history of the button's logic voltage levels, in memory. The trick to detect a button press, then, is just to recognize which specific histories correspond to which button events (press or release) or which long-run states (up or down). That's why I think of this as being a "pattern matching" debounce routine at heart, but I don't think that [Jack] knew it at the time.

The nice thing about this algorithm is that is splits neatly into two parts: an updater that stores the button's history, and various analysis functions that look for their corresponding patterns. Our API will look something like this:

```

void update_button(uint8_t *button_history);
uint8_t is_button_up(uint8_t *button_history);
uint8_t is_button_down(uint8_t *button_history);
uint8_t is_button_press(uint8_t *button_history);
uint8_t is_button_release(uint8_t *button_history);
uint8_t read_button(void);

```

Your code just has to call the update function periodically — anything from every millisecond to every ten milliseconds is probably OK. It's best if it's actually regular, and the update function is short enough that you could call it from a system tick routine.

Then, when you want to know what the button's up to, you just call the respective functions and they return a one or zero. The way I'm implementing it, the button's history is defined in the main routine and then passed to the various button routines by pointer, but this is a matter of choice. If you're implementing many buttons, or with a strange architecture, you're going to change this stuff anyway.

If you're not using an 8-bit microcontroller, for instance, these routines work even better with a 16-bit or 32-bit wide history.

I'm also assuming that you've got a `read_button()` function that returns a one if the button pin currently has the voltage on it that corresponds to a button press, and a zero otherwise. In AVR C, this is as simple as `( (PORTD & (1<<PD2)) == 0 )` for instance. On an Arduino, this would be `digitalRead(pin) == 0;`.

Storing the History

First let's consider the updater. A button has only two electrical states: high and low voltage. It may be bouncing between them over time, but we can think of its state as being one bit. In true embedded programming form, we're going to store each of these bits one at a time in a byte (or word). Using bit-shift operations, this turns out to be exceptionally easy, and it'll run ridiculously fast on most microcontrollers.

```
void update_button(uint8_t *button_history){
    *button_history = *button_history &lt;&lt; 1;
    *button_history |= read_button();
}
```

We're only doing the minimum of what we need to here. The updater function takes (a pointer to) the button history, and the first line is a bit-shift, which rolls in a zero from the right-hand side. If `history = 0b00110101` before the shift, `history = 0b01101010` after. The result of our `read_button()` function is written into the newly-opened first bit position. That's all there is to it.

Compared to the tangled web of `if...then` statements in the delay debounce's update function, this one is blindingly fast. It's short enough that you can incorporate it directly into a system-tick interrupt routine if you want to. The rest of the job is interpreting the bit sequences, and that's where the secret sauce is located.

Finding the Button Press

So we've got this constantly updated stream of ones and zeros flying past. [Jack]'s idea was to look for the bit pattern that corresponds to a button press. Without giving too much away, this is going to reduce the problem of detecting our various button states to simply comparing `button_history` to a specific value.

Imagine the bitstream that corresponds to a non-bouncing button press: `...00000000111111111111...` and so on. You could test for a bitstream like `00001111` and you'd detect the press. But think about how a bouncing button press looks: `...0000100101111111111....` The crucial eight bits are `01111111` — the last bit of the bounce followed by the solid stream of pressed values. Here's the 8-bit version of [Jack]'s button-press detector:

```
uint8_t is_button_pressed(uint8_t button_history){
    return (button_history == 0b01111111);
}
```

Wow, that's simple — the entire routine just checks to see if `button_history` is `0b01111111`. Detecting a release is then a test for `0b10000000` and the up and down states are `0b00000000` and `0b11111111` respectively. The memory overhead is small, all of the functions are essentially one-liners, and the update and test functions are separable. There's no need to keep track of what state the button is in, or so it seems, because `0b01111111` already contains the notion of a state change between unpressed and pressed. And it works surprisingly well. At least for the specific purpose of debouncing.

But I've got some janky buttons in my junk drawer that revealed a subtle flaw in the logic. For instance, I have one button that will occasionally glitch while you're holding it down. It emits bit patterns that look like `...11110111111...` even though I'm holding the button solidly down. Worse, it'll do that a few tens to hundreds of milliseconds after the button press took place.

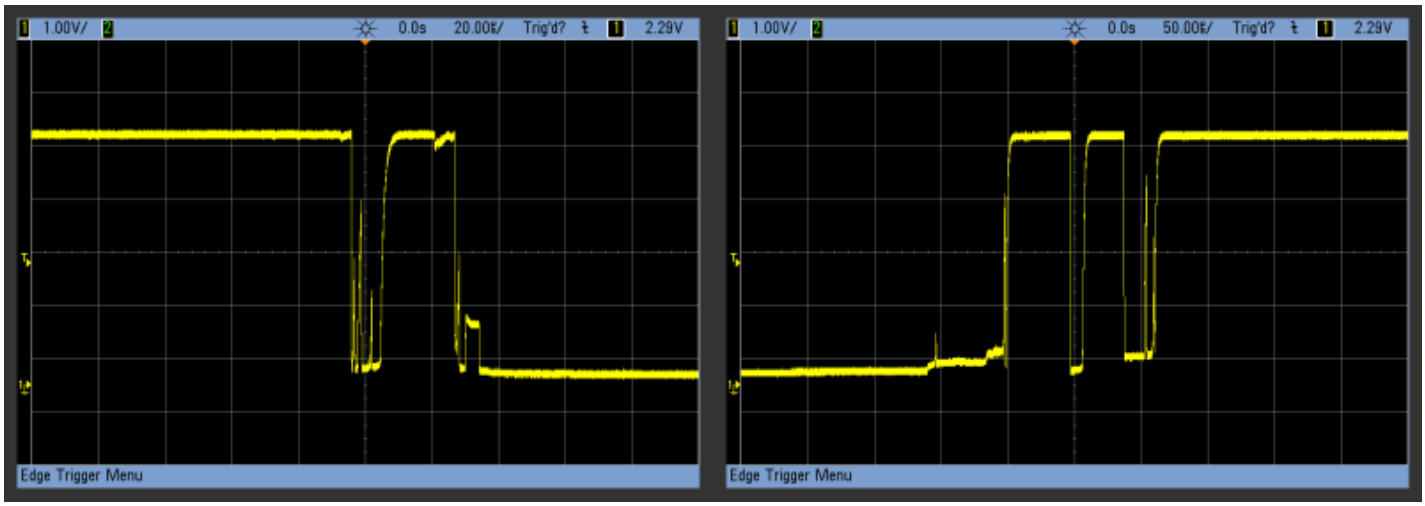
My intermittent button triggers a spurious button press event in [Jack]'s debouncer. A single zero in the bitstream arguably shouldn't retrigger a button press event, but with this algorithm it invariably does. Similarly, a single high-voltage spike would insert a one into the history, resulting in a false double-release, but I think

that's less likely.

And note that the delay debouncer wouldn't fall for this. It keeps track of the button's state, and only tests for a button press event when it's in the "up" state. Our glitchy `1111111011111` wouldn't fool it because the button would already be in the "down" state, and you can't press a button that's already down. Indeed, the delay debouncer, aside from the code bloat, seems to be very robust because of its internal state machine. It's not a bad method at all.

The Ultimate Debouncer(tm)

But we can do better. The trick is to take the essential logic from [Jack]'s debouncer but improve on the detection function. That actually shouldn't be hard, because all [Jack]'s test is doing is differencing the data — testing for a change that persists. But what would happen if we looked at the entirety of the bounce?



People are great pattern recognizers. What do you see in the bounce graphs? You see three regimes — one where the voltage is constant, one where it's wiggly, and one where it's constant again, but opposite of the initial state. Let's build on that; we'll test for a period of highs, then throw away some of the junk in the middle, and then test for a period of lows. Depending on where we sampled it, the example button press might look like `111110100000` or `111110110000`. Both of these match the 8-bit value `11xxx000`, where `x` represents a don't-care bit. Similarly, the release here is something like `0000011011111` or `000001010111111` or similar. These will match `00xxx111`, with the advantage that our intermittent glitch sequence `1111110111111` will never match it. Now we're getting somewhere. (OK, a two-period intermittent *will* match — either update more slowly or store more bits of history.)

We're not done yet, though. Once we've detected a press or release event, we need to clear out the don't-care section. Otherwise, we'll detect multiple presses. For instance a no-bounce transition would trigger four times as it moves through the history: `00000111`, `00001111`, `00011111`, and `00111111` all correspond to valid press events. So we'll need to reset the history once a valid press is detected. And that's as easy as resetting it to `11111111` — the history that corresponds to the "down" state, which makes logical sense after the button has been pressed.

In addition to preventing (false) re-triggering, this over-writing of the history buys us a bit of hysteresis. Beforehand, there could be bouncy zeros hiding in any of the don't-care bits, waiting to trip us up in short order. But now that everything is written to all ones, it's going to take a little bit more time for a zero to get over into the left-hand side. We've made it easier for a button release event to be detected after a press, and that's just what we want.

Implementation

The rest of the debouncer is details. I'll implement the don't care region by setting the middle bits to zero in the comparison and then testing for zeros there. Resetting the button history will require that our testing functions are able to write the button history as well as read it, suggesting that the history is passed into those functions by pointers again.

Here, for instance is a function to test for a button press:

```

#define MASK   0b11000111
uint8_t is_button_pressed(uint8_t *button_history){
    uint8_t pressed = 0;
    if ((*button_history &amp; MASK) == 0b00000111){
        pressed = 1;
        *button_history = 0b11111111;
    }
    return pressed;
}

```

Compared to [Jack]'s detection code, it looks a lot more complicated, but it's actually only a few cycles slower. The function takes in (a pointer to) the button history, and then logically-ANDs it with our mask. That has the effect of saving each bit where the mask is a one, and zeroing each bit where the mask is a zero, but it does it very efficiently on a microcontroller.

This altered history value is compared with a number that has zeros at the front, zeros in the don't-care region, and ones at the back. This corresponds exactly to our idea of a pre-bounce, bounce, and post-bounce split of the bitstream. Last but not least, we re-write the button history when the press is detected, and return a one.

Just for completeness, here's the rest of the detection code, although you're not going to be surprised by any of it. Put together, this code is only marginally slower than [Jack]'s blindingly-fast debouncing, and it provides a degree of noise immunity that his code doesn't.

```
uint8_t is_released(uint8_t *button_history){
        uint8_t released = 0;
        if (mask_bits(*button_history) == 0b11000000){
                released = 1;
                *button_history = 0b00000000;
        }
        return released;
}
uint8_t is_button_down(uint8_t *button_history){
        return (*button_history == 0b11111111);
}
uint8_t is_button_up(uint8_t *button_history){
        return (*button_history == 0b00000000);
}

```

And here's how you'd use it:

```
int main(void)
{
        uint8_t button_history=0;
        uint8_t press_count=0;
        uint8_t release_count=0;
        while (1) {
        do_important_stuff();
        update_button(&amp;button_history);
        if (is_button_pressed(&amp;button_history)){
                        press_count++;
        }
        if (is_button_released(&amp;button_history)){
                        release_count++;
        }
        if (is_button_down(&amp;button_history)){
            light_LED_or_something();
        }
    }
}
```

The advantage of separating the detection functions from the update function is that you don't need to test for states or events that you don't care about, unlike the delay debouncer which relies on knowing which state it's in and testing for all possible transitions. This makes it a lot leaner and faster, and you only use what you need. On the other hand, if you only ever wanted to test for button pushes, you could really streamline the whole thing into one function, like so:

```
uint8_t test_for_press_only(void){
    static uint8_t button_history = 0;
    uint8_t pressed = 0;
    button_history = button_history &lt;&lt; 1;
    button_history |= read_button();
```

```
    if (button_history &amp; 0b11000111 == 0b00000111)){
        pressed = 1;
        button_history = 0b11111111;
    }
    return pressed;
}
```

How's that for efficient? No arguments, no pointers, completely self-contained and it compiles to around twenty machine instructions. Just make sure you call it every five to ten milliseconds and you're done: debounce and de-glitch in one function. Bam! It's even short enough that I wouldn't hesitate to call it within a millisecond-tick-style interrupt routine.

Tuning

With the delay debouncer, you had to decide how long to wait for the bouncing to settle down, and that also affected how quickly the microcontroller could react to a button press. So while you might be tempted to delay for 100ms, the user would notice that your device feels sluggish. Something in the five to twenty millisecond range is probably about right.

The same goes for the Ultimate Debouncer(tm). In the 8-bit version above, we're allowing three don't-care bits for the bouncing, so if a worst-case bounce takes ten milliseconds or so, we'll want an update rate of once per five milliseconds to allow for some margin. Since we're requiring three good observations after the bounce window, this routine will react in a minimum of fifteen milliseconds plus the duration of the actual bouncing. Less frequent updating will increase noise and bounce immunity, but decrease response time, naturally.

If you're using sixteen or more bits for the history, for instance on a more modern microcontroller, there's a lot more room to play around with the ratio of pre-bounce, bounce, and post-bounce timings. For instance, `0b00000xxxxx111111` is broadly similar to what we've used here, but `0b00xxxxxxxx11111` allows for a much longer bounce period, proportionally. By trading off left-hand-side bits versus don't-care bits, you're trading glitch immunity against longer bounces. In practice, it's unlikely that you're going to have to worry about any of this optimization, but it's nice to know you could.

Conclusion

So is this truly the Ultimate Debouncer(tm)? I really like it, and the glitch-immunity has demonstrably proven its value on at least one intermittent-prone button that I own. It can be calibrated to be as discriminating as the more common delay-style debouncers, but it responds faster and requires less memory and fewer CPU cycles. And your CPU has better things to do than debounce buttons.

Once you've worked through the code, it's not so bad, but I'll admit that the binary constants are a little bit unreadable at first. I don't much like code that relies on comments to be readable, and these functions are pushing too far in that direction.

But don't take my word for it. Try it out, and let me know what you think.